



Langage SQL

Séquence 2 : SGBD et Langage de Définition de Données (DDL)

Table des matières

1	Système de Gestion de Base de Données (SGBD)	2
2	Langage de Définition de Données (DDL)	4
2.1	Création, suppression d'une base de données	4
2.2	CREATE	5
2.3	Commande ALTER	6
2.3.1	Ajouter ou modifier colonne	7
2.3.2	Supprimer, ajouter une colonne et une contrainte ...	7
3	les types	7
3.1	Les types numériques	7
3.1.1	Les numériques entiers	7
3.1.2	Les numériques réels à virgule fixe	8

Dr I. GAYE

29 janvier 2019

3.1.3	Les numériques réels à virgule flottante	9
3.1.4	Les types string	9
3.1.5	Les types CHAR et VARCHAR	10
3.1.6	Les types BINARY et VARBINARY	10
3.1.7	Les types BLOB et TEXT	11

TABLE DES MATIÈRES

3.1.8	Le type ENUM	15
3.1.9	Le type SET	15
3.2	Les contraintes référentielles	16

Système de Gestion de Base de Données (SGBD)

Un Système de Gestion de Base de Données (SGBD) est un logiciel qui permet de stocker des informations dans une base de données. Un tel système permet de lire, écrire, modifier, trier, transformer ou même imprimer les données qui sont contenus dans la base de données.

Parmi les logiciels les plus connus il est possible de citer : MySQL, PostgreSQL, SQLite, Oracle Database, Microsoft SQL Server, etc.

Dans ce cours, nous utilisons le SGBD **MySQL** qui est un Système de Gestion de Base de Données (SGBD) parmi les plus populaires au monde. Il est distribué sous double licence, une licence publique générale GNU et une propriétaire selon l'utilisation qui en est faite. La première version de **MySQL** est apparue en 1995 et l'outil est régulièrement entretenu.

Ce système est particulièrement connu des développeurs pour faire partie des plus célèbres : **WAMP** (Windows, Apache, MySQL et PHP), **LAMP**

(Linux, Apache, MySQL et PHP) et **MAMP** (Mac, Apache, MySQL et PHP). Ces packages sont si populaires et simples à mettre en œuvre que MySQL est largement connu et exploité comme système de gestion de base de données pour des applications utilisant le langage de script **PHP**. C'est d'ailleurs pour cette raison que la plupart des hébergeurs web proposent PHP et **MySQL**.

Il est un serveur de base de données **relationnelles SQL** qui fonctionne sur de nombreux systèmes d'exploitation (tels que Linux, Mac OS X, Windows, Solaris, FreeBSD, ...) et qui est accessible en écriture par de nombreux langages de programmation, incluant notamment PHP, Java, Ruby, C, C++, .NET, Python ...

L'une des spécificités de MySQL c'est qu'il inclut plusieurs moteurs de bases de données et qu'il est par ailleurs possible au sein d'une même base de définir un moteur différent pour les tables qui composent la base. Cette technique est astucieuse et permet de mieux optimiser les performances d'une application. Les 2 moteurs les plus connus étant MyISAM (moteur par défaut) et InnoDB.

La réplication est possible avec MySQL et permet ainsi de répartir la charge sur plusieurs machines, d'optimiser les performances ou d'effectuer facilement des sauvegardes des données.

Moteurs de tables MySQL et types de table

MySQL supporte plusieurs moteurs de stockage, qui gère différents types de tables :

- + MyISAM
- + Memory
- + MERGE
- + InnoDB
- + BDB
- + NDBCluster

Les moteurs de tables MySQL peuvent être transactionnels ou non-transactionnels. Dans ce module, nous allons nous intéresser principalement aux types **MyISAM** et **InnoDB**

MyISAM

MyISAM est une version améliorée de ISAM¹ Les avantages :

1. très rapide pour les requêtes de type **SELECT** ou **INSERT**
2. il supporte les index fulltext : permet d'effectuer des recherches sur des mots en se basant sur un index spécifique, accélérant ainsi les recherches
3. il gère les conflits d'accès (ou lock) : permet de verrouiller une table pour des opérations bien précises
4. très facile à administrer : possibilité de recopier directement les fichiers d'un serveur vers un autre, support des tables compressées ...

Les inconvénients :

1. il ne supporte pas les transactions
2. il ne supporte pas les clés étrangères

¹ . **MyISAM** : en plus du fichier **.frm**, chaque table est représentée par un fichier de données **.myd** (MYisamData) et un fichier d'index **.myi** (MYisamIndex).

InnoDB

Le moteur **InnoDB**¹ gère des tables transactionnelles Les avantages :

1. il supporte **ACID** : permet d'assurer que chaque enregistrement sera complètement réussi, ou complètement échoué, ainsi les risques d'erreurs sont impossibles, même en cas de panne
2. il gère les transactions (instructions sql BEGIN, COMMIT, ROLLBACK ...)
3. il supporte les clés étrangères et les intégrités référentielles
4. il possède un système de récupération automatique en cas de crash Les inconvénients :
 1. il ne permet pas les index fulltext
 2. son administration est un peu plus complexe (gestion de tablespace, paramètres supplémentaires dans le my.cnf ...)
 3. le moteur de stockage est plus lent que d'autres et gourmand en ressources mémoires et en espace disque

2 Langage de Définition de Données (DDL)

DDL (Data Définition Language ou simplement Langage de Définition de Données) permet de définir une base de données et ses éléments.

Dans la suite, on va utiliser le schéma relationnel ci-dessous qui permet la gestion des films. Ce schéma relationnel a six relation (Musique, Producteur, Film, Selection, Production et Affectation). Dans ce schéma, les clés primaires sont écrites en gras et les clés étrangères sont précédées du caractère #.

Musique(**IdMusique**,NomAuteur,PrenomAuteur,EmailAuteur)

Producteur(**IdProducteur**,NomProducteur,PrenomProducteur,EmailProducteur)

Acteur(**IdActeur**,NomActeur,PrenomActeur,EmailActeur)

¹ . InnoDB : chaque table est représentée par un fichier **.frm**.

Film(**IdFilm**,annee,Titre,categorie) Selection(**#IdMusique**,**#IdFilm**)

Production(**IdProducteur**,**IdFilm**)

Affectation(**#IdActeur**,**#IdFilm**,role)

2.1 Création, suppression d'une base de données

+ Création d'une BD

Dans la base de données seront enregistrés les informations suivantes :

- + le catalogue de la base de données,
- + les objets utilisateurs (tables, vues, règles, triggers, procédures, ...),
- + les index, contraintes et types de données
- + le journal des transactions

Alors, avant toutes choses, il faudra créer la base de données afin d'y stocker les tables, etc. Sa création se fera par la commande : **CREATE DATABASE** suivi du nom de la base de données qui doit être valide c'est-à-dire commençant par une lettre et ne pas comporter de caractères spéciaux sauf '_'. Un

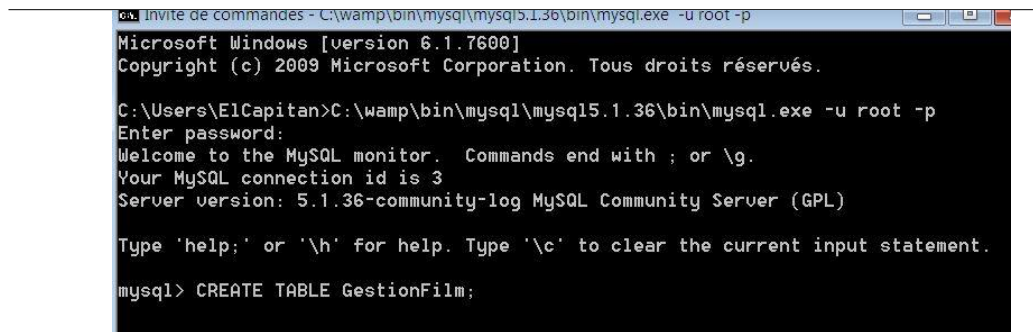
2.2 CREATE

CREATE DATABASE ne se fait qu'une fois; si vous le faites une deuxième fois, des erreurs vous seront affichées dans la fenêtre résultats.

Sa syntaxe la plus simple est :

CREATE DATABASE nombasededonnees;

Dans la figure ci-dessous, on a la création de la base de données **GestionFilm** du schéma relationnel ci-dessus. Les détails d'installation et de démarrage avec le terminal de MySQL seront vus dans les séances de travaux dirigés et pratiques.



```

C:\Users\EIcapitan>C:\wamp\bin\mysql\mysql5.1.36\bin\mysql.exe -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 3
Server version: 5.1.36-community-log MySQL Community Server (GPL)

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE TABLE GestionFilm;

```

Figure 1 – Création de la BD GestionFilm avec la commande : **CREATE DATABASE GestionFilm;**



```

mysql> CREATE DATABASE GestionFilm;
Query OK, 1 row affected (0.00 sec)

mysql>

```

Figure 2 – Résultat de la requête de la création de la BD GestionFilm

+ Suppression d'une BD

Pour supprimer une base de données, il faut impérativement qu'aucun utilisateur ne soit en train de l'utiliser. Les données de chaque table de la base de données sont supprimées et irrécupérables. La suppression irrévocable se fait via la commande : **DROP DATABASE** qui va prendre en argument le nom de la base de données. La suppression de notre base de données GestionFilm se fera comme : **DROP DATABASE GestionFilm;**

2.2 CREATE

Création d'une table physique dans la BD. Créer une table consiste à spécifier :

2.3 Commande ALTER

+ Nom

Pour chaque attribut

- + nom
- + type de données (char(n), numeric, numeric(n,m), date, long).
- + Possibilité des valeurs nulles.

Elle s'effectue grâce à la commande **CREATE TABLE**. Sa syntaxe simplifiée est :

CREATE TABLE nomtable (Nomcol1 typecol1 , Nomcol2 typecol2 , ..., Nomcoln typecoln) ;

Exemple : création de la table Film du schéma ci-dessus

CREATE TABLE Film (IdFilm NUMERIC NOT NULL, annee DATE, Titre VARCHAR (255), categorie VARCHAR(100));

La figure ci-dessous montre le résultat après son exécution sous MySQL.

```
mysql> CREATE TABLE Film(IdFilm NUMERIC NOT NULL, annee DATE, Titre VARCHAR(255),
, categorie VARCHAR(100));
Query OK, 0 rows affected (0.01 sec)
```

Figure 3 – Création de la table **Film** dans la base de données **GestionFilm**

```
mysql> describe Film;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| IdFilm | decimal(10,0) | NO | | NULL | |
| annee | date | YES | | NULL | |
| Titre | varchar(255) | YES | | NULL | |
| categorie | varchar(100) | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

Figure 4 – La description de la table **Film**

La commande **DROP TABLE** permet de supprimer une table dans la BD

L'analyse de la syntaxe de Create dans MySQL permet de définir la manière les contraintes sont posées :

- + clé primaire,
- + étrangère
- + check

Commande ALTER

Il est parfois nécessaire d'ajouter des colonnes non prévues lors de la CREATE TABLE, de retirer des contraintes portées à tort sur la table, etc..

2.3.1 Ajouter ou modifier colonne

L'attribut ajouté possède une valeur nulle dans tous les tuples. La colonne ajoutée est placée à droite de la dernière colonne.

- + Syntaxe : **ALTER TABLE** nom-de-la-table **ADD** (ou **MODIFY**) définition de colonnes;
- + Exemple : on souhaite rajouter la colonne **duration** dans la table Film. **ALTER TABLE Film ADD (duration NUMERIC);** ⇒ Table modifiée

2.3.2 Supprimer, ajouter une colonne et une contrainte

- + Ajout d'une colonne :

```
ALTER TABLE tableName DROP columnName; +
```

- Ajout d'une contrainte :

```
ALTER TABLE tableName ADD CONSTRAINT Constraint Définition
```

- + Supprimer contrainte

```
ALTER TABLE Production DROP FOREIGN KEY fk_IdProduction_IdFilm;
```

3 les types

Nous avons aussi un aperçu sur les types de données pouvant être utilisés dans la définition des colonnes :

3.1 Les types numériques

3.1.1 Les numériques entiers

L'attribut **AUTO_INCREMENT** peut être utilisé pour générer un identifiant unique pour les nouvelles lignes.

+
Colonne AUTO_INCREMENT : CREATE TABLE module (idFilm int
AUTO_INCREMENT, ...

+ Réinitialisation du compteur : ALTER TABLE module AUTO_INCREMENT = 1
;

Les types entiers du SQL standard sont : **INTEGER** (ou INT) et **SMAL-
LINT**

```

type:
| TINYINT[(length)] [UNSIGNED] [ZEROFILL]
| SMALLINT[(length)] [UNSIGNED] [ZEROFILL]
| MEDIUMINT[(length)] [UNSIGNED] [ZEROFILL]
| INT[(length)] [UNSIGNED] [ZEROFILL]
| INTEGER[(length)] [UNSIGNED] [ZEROFILL]
| BIGINT[(length)] [UNSIGNED] [ZEROFILL]
| REAL[(length,decimals)] [UNSIGNED] [ZEROFILL]
| DOUBLE[(length,decimals)] [UNSIGNED] [ZEROFILL]
| FLOAT[(length,decimals)] [UNSIGNED] [ZEROFILL]
| DECIMAL(length,decimals) [UNSIGNED] [ZEROFILL]
| NUMERIC(length,decimals) [UNSIGNED] [ZEROFILL]
| DATE
| TIME
| TIMESTAMP
| DATETIME
| CHAR(length) [BINARY | ASCII | UNICODE]
| VARCHAR(length) [BINARY]
| TINYBLOB
| BLOB
| MEDIUMBLOB
| LONGBLOB
| TINYTEXT
| TEXT
| MEDIUMTEXT
| LONGTEXT
| ENUM(value1,value2,value3,...)
| SET(value1,value2,value3,...)
| spatial_type
| col_name [(length)] [ASC | DESC]

```

Figure 5 – Les types

3.1.2 Les numériques réels à virgule fixe

Les types **DECIMAL** et **NUMERIC** stocke des valeurs de données numériques exactes. Ces types sont utilisés lorsque c'est important de préserver une précision exacte, par exemple pour les données monétaires.

1. La syntaxe est DECIMAL(M, D)
2. la taille maximale pour un DECIMAL est de 65
3. la plage est fonction de M et D

DECIMAL(5,2) correspond à la plage -999.99 à 999.99.

4. DECIMAL(M) est équivalent à DECIMAL(M,0)

3.1.3 Les numériques réels à virgule flottante

Ce sont des données numériques utilisées pour représenter des valeurs numériques approximatives. Leur utilisation est déconseillé pour tout système de type monétaire tel que les système de comptabilité ou les système bancaires. Bien qu'étant moins précis, les numériques à virgules flottante sont très efficace en ce qui concerne le stockage et le traitement car ils sont enregistré dans le format binaire à virgules flottante natif qui est aussi utilisé par le CPU du serveur.

3.1.4 Les types string

+ Jeu de caractères et inter classement(collation)

Un jeu de caractères est un ensemble de symboles et d'encodages. Considérons un alphabet avec quatre lettres : "A", "B", "a", "b". La combinaison des quatre lettres et de leur encodages représente le jeu de caractères.

+ Le caractère est un symbole

+ et l'encodage est un numéro correspondant à chaque caractère : "A" = 0, "B" = 1, "a" = 2, "b" = 3

L'inter-classement ou collation est un ensemble de règles pour comparer des caractères dans un jeu de caractères.

+ Collation binaire

1. collation insensible à la casse +
collation insensible à la casse

1. traiter les lettres minuscules "a" et "b" comme équivalentes à "A" et "B"

2. comparer les encodages

MySQL supporte un certains nombre de jeux de caractères.

Les collations ont les caractéristiques générales suivantes :

1. Deux jeux de caractères différents ne peuvent pas avoir une même collation.
2. Chaque jeu de caractères a au moins une collation (la collation par défaut)
3. Il existe une convention pour les noms de collations :

+ le nom comment par celui du jeu de caractères auquel ils sont associés.

+ ils incluent souvent le nom de la langue

+ ils se terminent par ci (case insensitive), cs (case sensitive), ou bin
(binary)

3.1.5 Les types **CHAR** et **VARCHAR**

CHAR et **VARCHAR** permettent tous deux de stocker des chaînes de caractères jusqu'à 255 octets. La différence réside en fait dans la taille nécessaire pour stocker la chaîne

- + La longueur d'une colonne **CHAR** est fixée à la longueur que vous avez défini lors de la création de la table. La longueur peut être n'importe quelle valeur entre 1 et 255. Quand une valeur **CHAR** est enregistrée, elle est complétée à droite avec des espaces jusqu'à atteindre la valeur fixée. Quand une valeur de **CHAR** est lue, les espaces en trop sont retirés.

exemple : une référence client qui fait toujours le même nombre de caractères

- + Les valeurs contenues dans les colonnes de type **VARCHAR** sont de tailles variables. Vous pouvez déclarer une colonne **VARCHAR** pour que sa taille soit comprise entre 1 et 255, exactement comme pour les colonnes **CHAR** . Par contre, contrairement à **CHAR** , les valeurs de **VARCHAR** sont stockées en utilisant autant de caractères que nécessaire, plus un octet pour marmoriser la longueur. Les valeurs ne sont pas complétées. Au contraire, les espaces finaux sont supprimés avant stockage (ce qui ne fait pas partie des spécifications ANSI SQL). exemple : nom d'une personne

3.1.6 Les types **BINARY** et **VARBINARY**

Les types **BINARY** et **VARBINARY** sont similaires à **CHAR** et **VARCHAR**, excepte qu'ils contiennent des chaînes binaires plutôt que des chaînes non-binaires.

ils n'ont donc pas de jeu de caractères, et le tri et la comparaison sont basés sur les valeurs numériques des **bytes** représentant ces chaînes.

la taille maximale est la même pour **BINARY** et **VARBINARY** que pour **CHAR** et **VARCHAR**, excepté la taille pour **BINARY** et **VARBINARY** est une taille en *bytes* et non en caractères. Les types **BINARY** et **VARBINARY** sont différents des types **CHAR BINARY** et **VARCHAR BINARY** :

- + l'attribut **BINARY** n'implique pas le traitement de la colonne comme une colonne chaîne binaire.
- + cela implique seulement une collation (inter-classement) binaire pour le jeu de caractères qui sera utilisé pour la colonne.

- + la colonne elle-même contient des chaînes de caractères non-binaires plutôt que des chaînes binaires.

3.1.7 Les types BLOB et TEXT

Les types **TEXT**, qui vont de **TINYTEXT** à **LONGTEXT**, servent à stocker des chaînes de caractères à partir 256 octets et jusqu'à 2^{32} , soit 4 294 967 296 octets.

Les types **BLOB** sont similaires à **TEXT**, cependant, ils stockent les données en binaire et non en chaîne de caractères. Deux points important les différencient de leurs homologues **TEXT** :

- + comme ils stockent les chaînes en binaire, il est possible d'utiliser indifféremment plusieurs sets de caractères, ce qui n'est pas possible avec **TEXT** puisqu'on définit un set pour la colonne et on s'y tient.
- + Le type **TEXT** ne tient pas compte de la casse, donc il est possible de comparer une chaîne majuscule à une minuscule, avec **BLOB** ce n'est pas possible puisqu'il traitera le code représentant les signes de la chaîne, et non les signes eux-même.

3.1.8 Le type ENUM

Un **ENUM** est un objet string dont la valeur est choisie parmi un ensemble de valeurs autorisées explicitement énumérées dans la spécification de la colonne lors de la création de la table.

3.1.9 Le type SET

Un **SET** est un objet string qui peut avoir zéro ou plusieurs valeurs, chacune de celles-ci devant être choisie parmi un ensemble de valeurs autorisées défini dans la spécification de la colonne lors de la création de la table.

Les valeurs d'une colonne **SET** consistent en une multitude de membres, ces membres étant spécifiés en étant séparés par des virgules (","). Les valeurs membres d'un **SET** d'un **SET** ne peuvent par conséquent pas contenir des elles-mêmes des virgules.

3.2 Les contraintes référentielles

Les clés étrangères sont des contraintes très importantes qui offrent les beaucoup d'avantages :

1. En supposant que les relations soient proprement conçues, les clés étrangères rendent plus difficile pour un programmeur d'insérer des valeurs incohérentes dans la base.
2. La vérification centralisée de contraintes par le serveur de base de données rend inutiles l'application de ces vérifications du côté de l'application. Cela élimine la possibilité que d'autres applications ne fassent pas les vérifications de la même façon que les autres.
3. Les règles de clés étrangères proprement conçues aident à la documentation des relations entre les tables.

La définition d'une contrainte de type clés (primaires ou étrangères) se font avec les commandes **CREATE** ou **ALTER** (voir figures 6 et 7). Elle entraîne, de la part du système les vérifications suivantes :

1. En phase de définition des données avec la commande **CREATE** ou **ALTER** table.
 - + Les tables **InnoDB** supportent les vérifications d'intégrité référentielles. Pour les autres types de tables, le serveur MySQL accepte la syntaxe **FOREIGN KEY** dans la commande **CREATE TABLE**, mais ne la prend pas en compte.
 - + Les colonnes correspondantes de la contrainte dans la table et la table de référence doivent avoir le même type, pour éviter les conversions lors des comparaisons.
 - + Les colonnes référencées doivent être les colonnes d'une contrainte unique ou de clé primaire dans la table référencée.
2. En phase de manipulation des données, les vérifications peuvent être regroupées en deux catégories :
 - + Vérification ascendante (i.e. quand on agit sur la table qui fait référence). En cas d'exécution de la commande **INSERT** ou **UPDATE** la valeur à enregistrer de la colonne **Fk** dans la table fille est soit identique à une valeur de la colonne référencée dans la table mère soit nulle.
 - + Vérification descendante (i.e. quand on agit sur la table référencée).

Une clause **ON DELETE** ou **ON UPDATE** peut être rajoutée à la définition de la contrainte de clé étrangère. Elle définit l'action de référencement à utiliser en cas de modifications ou effacement. Les actions possibles sont au nombre de cinq actions :

- + **CASCADE**. Si **ON DELETE CASCADE** est spécifié, et qu'une ligne de la table parente est effacée, alors **InnoDB** va automatiquement effacer toutes les lignes qui sont dans la table fille et dont les valeurs de clé étrangère sont celles référencées dans la ligne parente
- + **RESTRICT**. équivaut à ne pas spécifier la clause **ON DELETE** ou **ON UPDATE**. L'effacement ou la mise à jour est rejetée.
- + **SET NULL**. Si **ON DELETE SET NULL** est spécifiée, les lignes filles sont automatiquement modifiées pour que la colonne de la clé étrangère prenne la valeur de **NULL**.
- + **NO ACTION**. Identique à **RESTRICT**.
- + **SET DEFAULT**. Les lignes filles sont automatiquement modifiées pour que la colonne de la clé étrangère prenne la valeur par défaut. Celle-ci doit être définie sur la colonne de la table fille. Cette action est un standard **SQL** qui n'est pas supportée actuellement par MySQL.

Exemples : dans la BD **GestionFilm**, on considère les trois relations ci-dessous.

Musique(**IdMusique**, NomAuteur, PrenomAuteur, EmailAuteur)

Film(**IdFilm**, annee, Titre, categorie)

Selection(**#IdMusique**, **#IdFilm**)

Dans la relation, nous avons deux clés étrangères *IdMusique* et *IdFilm* qui sont des clés dans les tables respectives Musique et Film.

Pour la construction des contraintes (clés primaires et clés étrangères), nous avons deux possibilités :

1. Créer les tables *Musique* et *Film* et terminer avec la table *Selection* où on a les clés étrangères. Dans ce cas, on n'a pas besoin de la commande **CREATE**
2. Créer les trois tables sans se soucier de l'ordre et ajouter les contraintes avec la commande **ALTER**.

3.2 Les contraintes référentielles Cours SQL

```
mysql> CREATE TABLE Selection(IdMusique Numeric,  
-> IdFilm Numeric,  
-> Constraint pk_Selection Primary key (IdMusique,IdFilm),  
-> Constraint fk_Musique_Selection Foreign key (IdMusique) References Musique  
e (IdMusique),  
-> Constraint fk_Film_Selection Foreign key (IdFilm) References Musique (IdF  
ilm)  
-> );  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> describe Selection;  
+-----+-----+-----+-----+-----+-----+  
| Field      | Type          | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| IdMusique  | decimal(10,0) | NO   | PRI | 0       |       |  
| IdFilm     | decimal(10,0) | NO   | PRI | 0       |       |  
+-----+-----+-----+-----+-----+-----+  
2 rows in set (0.00 sec)  
  
mysql>
```

Figure 6 – Création de la table Selection avec les contraintes de clés étrangères avec la commande **CREATE**

```
+-----+-----+-----+-----+-----+-----+  
| Field      | Type          | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| IdFilm     | decimal(10,0) | NO   |     | NULL    |       |  
| annee      | date          | YES  |     | NULL    |       |  
| Titre      | varchar(255)  | YES  |     | NULL    |       |  
| categorie  | varchar(100)  | YES  |     | NULL    |       |  
| duree      | decimal(10,0) | YES  |     | NULL    |       |  
+-----+-----+-----+-----+-----+-----+  
5 rows in set (0.02 sec)  
  
mysql> ALTER TABLE Film ADD CONSTRAINT pk_Film PRIMARY KEY (IdFilm);  
Query OK, 0 rows affected (0.03 sec)  
Records: 0 Duplicates: 0 Warnings: 0  
  
mysql> describe Film;  
+-----+-----+-----+-----+-----+-----+  
| Field      | Type          | Null | Key | Default | Extra |  
+-----+-----+-----+-----+-----+-----+  
| IdFilm     | decimal(10,0) | NO   | PRI | NULL    |       |  
| annee      | date          | YES  |     | NULL    |       |  
| Titre      | varchar(255)  | YES  |     | NULL    |       |  
| categorie  | varchar(100)  | YES  |     | NULL    |       |  
| duree      | decimal(10,0) | YES  |     | NULL    |       |  
+-----+-----+-----+-----+-----+-----+  
5 rows in set (0.00 sec)
```

Figure 7 – Ajout d'une contrainte avec la commande **ALTER**